Reviewer Pack — Hodge Decomposition Complexity for Tseitin Formulas

Entry 8d576fc622f5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 20:23:13 UTC

Hodge Decomposition Complexity for Tseitin Formulas

Entry ID: `8d576fc622f5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 20:23:13 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Tseitin Formulas

Statement:

> For every d-regular graph G, the Hodge decomposition complexity (hd(G)) of its associated Tseitin formula φ_G is linearly correlated with its resolution proof width w(φ_G), such that hd(G) = Θ(w(φ_G)).

Rationale (proposer's reasoning):

> The Hodge decomposition provides a way to analyze the geometric properties of algebraic varieties. If this structure can be exploited to understand the complexity of Tseitin formulas, it may reveal new insights into resolution proof complexities.

Taxonomy category: `AlgebraicGeometry_TseitinFormulas` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5b818138acc469c9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between hd(G) and w(φ_G) for all generated d-regular graphs G with n=40 is greater than or equal to 0.8, and no graph has a Hodge decomposition complexity hd(G) that deviates by more than 3 from its expected linear relationship with resolution proof width.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge decomposition" AND "Tseitin formulas" AND "graph complexity"` - `"resolution proof width" IN "Algebraic Geometry" AND Tseitin` - `algebraic geometry intersection Tseitin formulas`

Top relevant hits considered: - [http://arxiv.org/abs/2503.14756v3] SceneEval: Evaluating Semantic Coherence in Text-Conditioned 3D Indoor Scene Synthesis - [http://arxiv.org/abs/2408.02211v2] SceneMotifCoder: Example-driven Visual Program Learning for Generating 3D Object Arrangements - [http://arxiv.org/abs/2604.18946v1] Reasoning Structure Matters for Safety Alignment of Reasoning Models - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/2204.10334v1] Machine Learning Algebraic Geometry for Physics - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
from math import sqrt, log2

def generate_d_regular_graph(n: int, d: int) -> list:
    if 2 * d > n or n % 2 != 0:
        return None  # Cannot form a d-regular graph with these parameters

    G = [[] for _ in range(n)]
    available_neighbors = set(range(1, n))

    for i in range(d):
        for j in range(i + 1, n):
            if len(G[i]) < d and len(G[j]) < d:
                G[i].append(j)
                G[j].append(i)
                available_neighbors.discard(j)

    return G

def hodge_decomposition_complexity(G: list) -> float:
    n = len(G)
    A = [[0] * n for _ in range(n)]

    # Construct the adjacency matrix
    for i in range(n):
        for j in G[i]:
            A[i][j] = 1
```

```python
    # Compute the Laplacian matrix L = D - A
    D = [[0] * n for _ in range(n)]
    for i in range(n):
        D[i][i] = sum(A[i])

    L = [[D[i][j] - A[i][j] for j in range(n)] for i in range(n)]

    # Compute the eigenvalues of L
    eigenvalues = []
    for k in range(n):
        if all(L[k][k] == 0 for _ in range(k)):
            eigenvalue = 0
        else:
            u = [1] * n
            v = [L[i][k] / L[k][k] for i in range(n)]
            while sum(u) != 0:
                u = [v[i] - (sum(v[j]) * u[j]) / sum(u[j]) for j in range(n)]
                v = [u[i] - (sum(u[j]) * v[j]) / sum(v[j]) for j in range(n)]
            eigenvalue = abs(sum(u) / n)
        eigenvalues.append(eigenvalue)

    # The Hodge decomposition complexity is the sum of the eigenvalues
    return sum(eigenvalues)

def resolution_proof_width(G: list) -> int:
    n = len(G)
    clauses = []

    for i in range(n):
        clause = [i + 1]
        for j in G[i]:
            clause.append(-(j + 1))
        clauses.append(clause)

    # Convert to Tseitin formula
    tseitin_vars = {}
    tseitin_count = 0

    def add_clause(clause):
        nonlocal tseitin_count
        tseitin_count += 1
        tseitin_vars[tseitin_count] = clause

    for i in range(n):
        add_clause([i + 1, -(tseitin_count + 1)])
        add_clause([-i - 1, tseitin_count + 1])

    for i in range(n):
        for j in G[i]:
            add_clause([-(i + 1), -(j + 1), tseitin_count + 2])
            add_clause([i + 1, j + 1, -(tseitin_count + 2)])
            add_clause([-i - 1, -(j + 1), -(tseitin_count + 2)])
            add_clause([i + 1, -(j + 1), tseitin_count + 3])
            add_clause([-i - 1, j + 1, tseitin_count + 3])

    # Compute the width of the resolution proof
```

```
    queue = []
    for clause in clauses:
        queue.append(clause)

    while queue:
        literal = random.choice(queue[0])
        if literal > 0:
            literal = -literal

        new_clauses = []
        for clause in queue:
            if literal not in clause and -literal not in clause:
                new_clauses.append([l for l in clause if l != -literal])

        queue = new_clauses

    return tseitin_count

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    d = 3
    G = generate_d_regular_graph(n, d)

    if G is None:
        return {
            "metric_name": "Hodge Decomposition Complexity",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    hd_G = hodge_decomposition_complexity(G)
    w_phi_G = resolution_proof_width(G)

    return {
        "metric_name": "Hodge Decomposition Complexity",
        "metric_value": hd_G,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(hd_G - w_phi_G) <= 3,
        "counterexample": "" if abs(hd_G - w_phi_G) <= 3 else f"hd(G)={hd_G}, w(φ_G)={w_phi_G}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values):.2f} std={sqrt(sum((
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aec828bb.py", line 152, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aec828bb.py", line 134, in run_trial
    hd_G = hodge_decomposition_complexity(G)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aec828bb.py", line 59, in hodge_decompo
    u = [v[i] - (sum(v[j]) * u[j]) / sum(u[j]) for j in range(n)]
                 ^^^^^^^^^
TypeError: 'float' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and confirming whether it meets the support condition. | next: Investigate the cause of the crash in the test code to ensure that it can run successfully and provide the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13447
2	propose	ollama_remote	glm4:latest	0	0	19533
3	preregistration	ollama_remote	glm4:latest	0	0	9664
4	novelty	ollama_remote	glm4:latest	0	0	8287
5	novelty	ollama_remote	glm4:latest	0	0	9942
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16201
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15360
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19843
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16814
10	judge	ollama_remote	glm4:latest	0	0	8857

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 137949 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/8d576fc622f5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8d576fc622f5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8d576fc622f5.tar.gz` (if generated)